

# Monitoreo de clúster HAProxy con Datadog

Juan David Cuero Reina, Juan Esteban Vila Martin, Alejandro Rodríguez, *Diego Alejandro Ramirez*

## I. INTRODUCCIÓN

Las arquitecturas basadas en contenedores y servicios distribuidos se han consolidado como el estándar para el despliegue de aplicaciones modernas gracias a su flexibilidad y eficiencia en el uso de recursos. En este contexto, la orquestación con tecnologías como Docker permite aislar procesos en entornos ligeros [4]. Sin embargo, administrar estas infraestructuras requiere implementar estrategias de balanceo de carga y monitoreo continuo para garantizar la alta disponibilidad y la tolerancia a fallos.

Este artículo presenta la implementación y análisis de un entorno virtualizado orientado al balanceo de carga y la observabilidad de servicios web. La infraestructura propuesta se despliega sobre máquinas virtuales controladas mediante Vagrant y VirtualBox, simulando un entorno de producción. Sobre esta base, se configura un nodo balanceador encargado de distribuir el tráfico hacia tres servicios *backend* independientes ejecutados en contenedores Docker [1].

Para evaluar el rendimiento del sistema, se integra la plataforma Datadog, logrando una observabilidad centralizada de métricas críticas como el consumo de CPU, memoria y el estado de los contenedores. Asimismo, se ejecutan pruebas de estrés mediante herramientas de generación de tráfico para identificar cuellos de botella y evaluar la estabilidad de la arquitectura bajo distintos volúmenes de solicitudes [5]. El resto de este documento está organizado de la siguiente manera: la Sección II aborda el contexto y el problema a resolver, mientras que la Sección III explora las alternativas de solución. La Sección IV describe el diseño de la arquitectura, seguida de su implementación en la Sección V. La metodología de pruebas y la discusión de los resultados obtenidos se presentan en las Secciones VI y VII, respectivamente. Finalmente, la Sección VIII expone las conclusiones derivadas de este trabajo.

## II. CONTEXTO Y PLANTEAMIENTO DEL PROBLEMA

El crecimiento exponencial de las aplicaciones web exige infraestructuras capaces de soportar altos volúmenes de tráfico garantizando una disponibilidad continua. En arquitecturas tradicionales, donde los servicios se ejecutan de manera monolítica o en servidores únicos, la escalabilidad, la tolerancia a fallos y la administración de recursos presentan limitaciones críticas.

Si bien la transición hacia arquitecturas basadas en contenedores resuelve los problemas de aislamiento y flexibilidad, introduce nuevas complejidades operativas. Cuando múltiples servicios *backend* operan de forma simultánea, surge el desafío de distribuir las peticiones de manera equilibrada para evitar la sobrecarga de nodos específicos. Asimismo, la naturaleza distribuida de los contenedores crea puntos ciegos operativos; sin herramientas

de observabilidad adecuadas, resulta sumamente complejo identificar latencias, picos de consumo de recursos o caídas de servicio en tiempo real.

Finalmente, la administración de estas infraestructuras suele carecer de validaciones preventivas. La ausencia de pruebas de carga dificulta determinar la capacidad real del sistema para prever su degradación, identificar cuellos de botella o garantizar los Acuerdos de Nivel de Servicio (SLA) ante picos repentinos de tráfico.

En consecuencia, el problema central radica en la necesidad de diseñar un entorno distribuido que no solo aisle los servicios, sino que integre eficientemente mecanismos de balanceo, observabilidad continua y validación de estrés operativo bajo condiciones de tráfico variables.

## III. ALTERNATIVAS DE SOLUCIÓN

La observabilidad de infraestructuras distribuidas puede abordarse mediante diversas plataformas. A continuación, se analizan las alternativas más relevantes, evaluando su idoneidad para un entorno compuesto por HAProxy y contenedores Docker.

### A. Prometheus y Grafana

Prometheus es un estándar *open-source* en entornos de contenedores que opera bajo un modelo *pull*. Combinado con Grafana, ofrece visualización avanzada y gran control sin costos de licencia. Para HAProxy, requiere exportadores específicos (*haproxy-exporter*). Su principal desventaja es la carga operativa: desplegar la pila completa (Prometheus, Alertmanager, Grafana, y almacenamiento a largo plazo) exige mantener múltiples componentes independientes, lo cual resulta ineficiente para despliegues de corto plazo o enfoques académicos [5].

### B. Zabbix

Plataforma *open-source* robusta y de larga trayectoria, ideal para infraestructuras tradicionales y redes empresariales. Sin embargo, su curva de aprendizaje es pronunciada y su integración nativa con arquitecturas *cloud-native* y HAProxy no es tan directa como la de herramientas modernas, requiriendo un esfuerzo de configuración manual que excede el alcance de este trabajo.

### C. New Relic y Dynatrace

Plataformas SaaS de observabilidad integral orientadas a nivel corporativo, destacadas por la detección de anomalías mediante Inteligencia Artificial. Aunque son sumamente potentes, sus modelos de licenciamiento y la complejidad de su configuración inicial representan barreras de acceso significativas para entornos académicos ágiles.

### D. Datadog (Alternativa Seleccionada)

Datadog es una plataforma SaaS que unifica métricas, *logs* y trazas bajo un único agente. Se seleccionó para esta implementación debido a tres ventajas concretas [5]:

1. **Integración nativa:** Permite recolectar automáticamente métricas de HAProxy (estado,

*backends*, latencia) mediante su configuración base (conf.yaml), sin necesidad de exportadores externos [5].

2. **Agente unificado:** Centraliza la recolección de métricas del sistema y *logs* de contenedores, eliminando la sobrecarga operativa de orquestar múltiples herramientas.
3. **Agilidad de despliegue:** Al ser SaaS, elimina la necesidad de alojar y mantener la infraestructura de monitoreo.

TABLA I. COMPARACIÓN DE PLATAFORMAS DE MONITOREO EVALUADAS

| Criterio              | Prometheus + Grafana        | Zabbix      | New Relic / Dynatrace | Datadog               |
|-----------------------|-----------------------------|-------------|-----------------------|-----------------------|
| Modelo                | Open source                 | Open source | SaaS comercial        | SaaS comercial        |
| Integración HAProxy   | Vía exporter                | Manual      | Limitada              | Nativa y documentada  |
| Curva de aprendizaje  | Alta (multi-componente)     | Alta        | Media                 | Baja                  |
| Recolección de Logs   | Requiere herramientas extra | Limitada    | Sí                    | Sí (Agente unificado) |
| Adecuación al trabajo | Media                       | Baja        | Media                 | Alta                  |

En conclusión, aunque Prometheus representa la alternativa *open-source* más cercana en capacidades, su sobrecarga operativa lo hace inviable para este contexto. Datadog ofrece el equilibrio óptimo entre potencia analítica, facilidad de despliegue e integración nativa con la infraestructura propuesta.

#### IV. DISEÑO DE LA SOLUCIÓN

La solución se fundamenta en una arquitectura distribuida de dos capas: una base de virtualización gestionada con Vagrant, y una capa superior de contenerización orquestada con Docker. Esta topología garantiza un entorno aislado, escalable e independiente del sistema operativo anfitrión.

A. **Infraestructura y Almacenamiento** El entorno base consta de dos máquinas virtuales con roles especializados. La primera es el nodo de cómputo principal, encargado de alojar la totalidad del clúster de contenedores.

La segunda actúa como nodo de almacenamiento. Para gestionar el ciclo de vida de los archivos, este nodo implementa un flujo dual: por un lado, cuenta con un servidor OpenSSH configurado con un entorno enjaulado (chroot) para recibir los archivos de forma segura vía SFTP desde los contenedores; por otro lado, ejecuta un servidor web NGINX nativo que expone el directorio de almacenamiento, permitiendo a los usuarios finales descargar los archivos mediante peticiones HTTP estándar [8].

B. **Capa de Balanceo y Lógica de Aplicación** Sobre el nodo de cómputo, el clúster opera dentro de una red interna aislada. El único punto de entrada es un balanceador de carga HAProxy, el cual distribuye el tráfico HTTP entrante utilizando el algoritmo Round Robin. Este componente incorpora mecanismos de health checks para excluir automáticamente nodos defectuosos y genera registros estructurados en formato JSON por cada transacción [1], [2].

El tráfico es distribuido hacia tres backends independientes. Estos consisten en aplicaciones web desarrolladas en Flask que procesan la carga y descarga de archivos. Cada nodo adjunta su identidad en la respuesta HTTP para evidenciar visualmente el balanceo y se comunica con el nodo de almacenamiento para persistir la información de manera centralizada.

C. **Capa de Observabilidad** Para la monitorización, se integra un contenedor dedicado con el Datadog Agent. Su diseño le permite descubrir dinámicamente los contenedores en ejecución y capturar métricas del sistema operativo anfitrión. La comunicación directa con HAProxy se establece mediante un UNIX Socket compartido, lo que habilita la extracción segura de telemetría nativa del balanceador sin requerir la apertura de puertos adicionales. Simultáneamente, el agente recolecta logs y rastros de rendimiento (APM), transmitiendo toda la información hacia Datadog Cloud [5].

D. **Capa de Generación de Tráfico** Para la validación bajo estrés, se define un contenedor basado en Artillery. Este componente simula la concurrencia de múltiples usuarios inyectando tráfico sintético hacia el balanceador, permitiendo evaluar el comportamiento de toda la arquitectura mediante escenarios predefinidos [7].

La topología completa se ilustra en la Fig. 1. El diagrama distingue los flujos principales: el tráfico HTTP del usuario hacia HAProxy y los backends, el flujo de persistencia vía SFTP y descarga vía NGINX, y finalmente, el canal lateral de telemetría hacia Datadog Cloud.

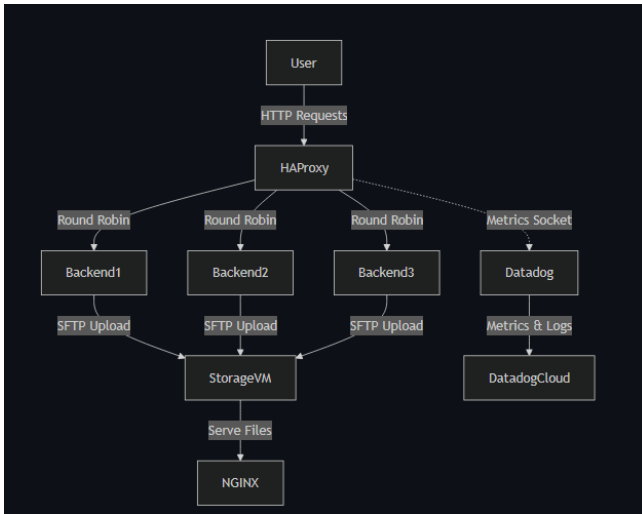


Fig. 1. Arquitectura general del sistema: clúster HAProxy, almacenamiento dual (SFTP/NGINX) y observabilidad centralizada.

## V. IMPLEMENTACIÓN

La implementación se estructura en cuatro etapas operativas: aprovisionamiento del entorno, orquestación del clúster de balanceo, configuración del almacenamiento y despliegue de la telemetría y pruebas.

### A. Aprovisionamiento del Entorno Base

El despliegue inicia mediante un Vagrantfile que define dos máquinas virtuales (Ubuntu 22.04 LTS). La MV lab (192.168.56.10) se aprovisiona mediante scripts/lab.sh, instalando Docker Engine y configurando permisos de ejecución sin superusuario. La MV storage (192.168.56.20) se prepara con scripts/storage.sh. Una vez listas, el clúster se levanta ejecutando docker-compose up --build -d desde el directorio compartido /vagrant, orquestando cinco servicios en la red interna cluster\_network [3], [4].

### B. Configuración de HAProxy y Backends

La lógica de HAProxy se define en haproxy.cfg, montado como volumen de solo lectura. El bloque global expone el socket de administración en /tmp/haproxy.sock, mientras que defaults formatea los logs transaccionales en JSON estructurado (capturando campos como client\_ip, backend y http\_status). El tráfico del puerto 8081 se deriva al bloque backend web\_back, el cual distribuye las peticiones mediante la directiva balance roundrobin. La resiliencia se garantiza configurando option httpchk GET /, permitiendo excluir nodos inactivos. Adicionalmente, se define el backend storage\_back, que registra el nodo de almacenamiento (storage\_node en 192.168.56.20:80) con health check activo, permitiendo a HAProxy supervisar la disponibilidad del servidor NGINX de

la VM storage. El panel de estadísticas se expone en el puerto 8080 (/stats) [2].

Los tres backends se compilan desde un Dockerfile (imagen python:3.9-slim). Consisten en aplicaciones Flask que exponen una interfaz HTML, utilizando la variable NODE\_ID para su identificación. Para garantizar la consistencia, montan el volumen local ./backends/uploads, compartiendo el acceso a los archivos procesados [9].

### C. Configuración de la MV Storage (SFTP y NGINX)

La MV storage opera con una función dual. Primero, recibe archivos de los backends vía SFTP; para ello, OpenSSH confina al usuario sftpuser mediante la directiva ChrootDirectory /sftp/uploads. Segundo, sirve dichos archivos instalando NGINX nativo. Su configuración (nginx.sftp.conf) utiliza un bloque location /files/ con la directiva alias hacia el directorio de subidas y habilita autoindex on. Esto permite descargar el contenido mediante URL directas (<http://192.168.56.20/files/>), las cuales son construidas dinámicamente por la interfaz de los backends [8].

### D. Integración del Agente Datadog

El contenedor de Datadog (Agente v7) inyecta su clave API vía un archivo oculto .env y apunta a us5.datadoghq.com. Mediante variables de entorno, se autoriza la recepción de tráfico no local (DD\_DOGSTATSD\_NON\_LOCAL\_TRAFFIC=true), se activa el rastreo APM (DD\_APM\_ENABLED=true) y se fuerza la captura integral de logs (DD\_LOGS\_CONFIG\_CONTAINER\_COLLECT\_ALL=true) [5].

El agente requiere volúmenes críticos del host para su operación: /var/run/docker.sock para auto-descubrimiento, /proc/ y /sys/fs/cgroup/ para métricas del sistema, y /var/lib/docker/containers para logs en crudo. La integración con el balanceador se logra mediante haproxy.d/conf.yaml, el cual instruye al agente a consumir la telemetría directamente desde el volumen compartido haproxy\_socket [5], [6]. Los dos monitores de alerta se provisionan importando los archivos datadog\_exports/monitorErrores.json y datadog\_exports/monitor\_backend\_caido.json directamente en la plataforma Datadog, eliminando la configuración manual.

### E. Escenarios de Artillery y Puesta en Marcha

El contenedor de Artillery monta ./artillery en /scripts. Contiene nueve escenarios YAML (normal, spike, soak, latency, mixed, error400, error500, traficoFlask, traficoNginx)

que varían la tasa de llegada (arrivalRate) y la duración para inducir estrés sobre el target `http://haproxy` [7].

El flujo completo de aprovisionamiento, despliegue y validación del clúster se sintetiza en los comandos de la Fig. 2

```
vagrant up && vagrant ssh lab
cd /vagrant && docker-compose up --build -d
# Validación del algoritmo Round Robin:
for i in {1..10}; do curl -s localhost:8081; done
docker-compose logs haproxy
```

**Fig. 2.** Secuencia principal de despliegue y validación.

## VI. PRUEBAS

La validación del sistema se estructuró en tres fases secuenciales sobre la MV lab, cubriendo: la correcta distribución de carga, la observabilidad bajo diversos patrones de tráfico y la resiliencia de la infraestructura.

### A. Verificación del Balanceo y Estado Base

Antes de inducir tráfico artificial, se estableció la línea base del clúster. La distribución *Round Robin* se comprobó inyectando diez solicitudes secuenciales (`for i in {1..10}; do curl -s localhost:8081; done`). Al rastrear la variable `NODE_ID` en las respuestas y auditar el panel de estadísticas en `http://192.168.56.10:8080/stats`, se confirmó la rotación equitativa entre los tres *backends*. Paralelamente, se validó en Datadog Cloud la correcta recolección en reposo de métricas del clúster importando el dashboard desde el archivo `datadog_exports/dashboard.json`. Este dashboard centraliza nueve widgets: peticiones por segundo global (`haproxy.frontend.requests.rate`), latencia promedio por backend (`haproxy.backend.response.time`), tasa de errores con series diferenciadas para 4xx y 5xx, consumo de RAM por contenedor (`container.memory.usage`), estado individual de backends (`docker.containers.running`), uso de CPU por contenedor (`container.cpu.usage`), conexiones por segundo (`haproxy.backend.session.rate`), distribución de respuestas exitosas vs errores en gráfico sunburst (2xx, 4xx, 5xx) y ancho de banda entrante y saliente en MB/s (`haproxy.backend.bytes.in_rate` y `out_rate`).

### B. Generación de Tráfico con Escenarios Artillery

Se diseñaron nueve escenarios de carga dirigidos al clúster dentro de `cluster_network`. Los siete escenarios principales apuntan al objetivo interno `http://haproxy:80`, mientras que `traficoFlask.yml` genera subidas reales de archivos a través de los backends y `traficoNginx.yml` genera descargas directamente contra la VM storage en `http://192.168.56.20`. Cada prueba fue orquestada individualmente mediante el comando `docker compose run --rm artillery run <nombre>.yml` [7].

La TABLA II detalla la configuración y el objetivo de observabilidad de cada escenario. Esta batería de pruebas permitió evaluar desde el comportamiento del sistema ante incrementos abruptos y cargas sostenidas, hasta la correcta captura de latencias inducidas y códigos de error (4xx y 5xx) por parte del agente de Datadog. Los escenarios `traficoFlask.yml` y `traficoNginx.yml` complementan la validación evaluando el flujo completo de archivos: subida vía Flask/SFTP y descarga vía NGINX, respectivamente.

**TABLA II.** ESCENARIOS DE PRUEBA ARTILLERY

| Escenario    | Duración | Tasa (usuarios/s) | Endpoint(s)                          | Objetivo Principal                  |
|--------------|----------|-------------------|--------------------------------------|-------------------------------------|
| normal.yml   | 30 s     | 5                 | GET /                                | Línea base de tráfico regular       |
| spike.yml    | 50 s     | 5 -> 50 -> 5      | GET /                                | Pico repentino de carga             |
| soak.yml     | 300 s    | 15                | GET /                                | Estabilidad bajo carga sostenida    |
| latency.yml  | 60 s     | 5                 | GET /, GET /slow                     | Impacto de nodos lentos en latencia |
| mixed.yml    | 30 s     | 20                | / (80%), /error (10%), /upload (10%) | Tráfico mixto y tasa de errores     |
| error400.yml | 20 s     | 10                | POST /upload (sin body)              | Generación de errores 400           |
| error500.yml | 20 s     | 10                | GET /error                           | Generación de errores 500           |

| Escenario        | Duración | Tasa (usuarios/s) | Endpoint(s)                 | Objetivo Principal                                   |
|------------------|----------|-------------------|-----------------------------|------------------------------------------------------|
| traficoFlask.yml | 60 s     | 10                | POST /upload (con archivo)  | Carga real de archivos vía Flask y SFTP              |
| traficoNginx.yml | 60 s     | 30                | GET /files/test.txt (NGINX) | Descarga real de archivos vía NGINX en la VM storage |

**C. Resiliencia y Alertas por Caída de Nodo** Para evaluar la tolerancia a fallos y la capacidad de reacción de la telemetría, se detuvo deliberadamente un contenedor (docker stop backend2) durante un ciclo de carga activa. El procedimiento validó tres comportamientos simultáneos: la redirección automática del tráfico por parte de HAProxy hacia los nodos sobrevivientes (gracias al *health check* option `httpchk GET / [2]`), la actualización del estado a *DOWN* en el panel de estadísticas local, y finalmente, la detección de la anomalía en Datadog, culminando en el disparo de la alerta configurada. Al reanudar el contenedor (docker start backend2), se confirmó su reincorporación automática al *pool* de balanceo.

## VII. DISCUSIÓN DE LOS RESULTADOS

Los resultados de la fase de validación permiten analizar el comportamiento del sistema desde tres dimensiones: efectividad del balanceo, rendimiento bajo estrés y escalabilidad arquitectónica.

### A. Desempeño del Balanceo Round Robin

La distribución equitativa de peticiones, previamente validada a nivel de logs y métricas de sesión, se reflejó empíricamente en el consumo de recursos de los backends. Como se observa en la Fig. 3. (Panel de Estado Individual), la carga de CPU se distribuyó de manera homogénea: 2.17% para backend3, 1.94% para backend2 y 1.44% para backend1. Estas ligeras variaciones responden a diferencias transitorias en el procesamiento del sistema operativo, confirmando la ausencia de asimetrías o acumulación de trabajo (bottlenecks) en la capa de aplicación bajo tráfico estable.

### B. Análisis de Telemetría bajo Estrés

La Fig. 3 consolida el comportamiento del clúster ante los escenarios generados con Artillery. Durante la inyección de tráfico degradado (latency.yml), el widget de latencia promedio registró un incremento transitorio hasta aproximadamente 1.6 unidades cerca de las 15:15. Tras finalizar la prueba, la métrica retornó inmediatamente a su línea base, demostrando que el sistema procesa eficientemente las colas sin acumular deuda técnica. El widget de conexiones por segundo (haproxy.backend.session.rate) permitió correlacionar en tiempo real el incremento de sesiones activas con los picos de tráfico inducidos por Artillery. El widget de tipo sunburst con las series 2xx, 4xx y 5xx ofreció una perspectiva proporcional de la distribución de respuestas, evidenciando el predominio de respuestas exitosas incluso durante los escenarios de error controlado. El escenario traficoFlask.yml validó el flujo completo de subida de archivos: la carga multipart fue procesada por los backends Flask y persistida en la VM storage vía SFTP, sin interrupciones del balanceador. El escenario traficoNginx.yml confirmó la capacidad de NGINX para servir descargas a 30 usuarios/s de forma estable, sin impacto en los backends Flask ni en el balanceador HAProxy. Finalmente, el widget de ancho de banda (haproxy.backend.bytes.in\_rate y out\_rate) confirmó que el volumen de datos transferidos se mantuvo dentro de los rangos esperados sin saturación de la interfaz de red del clúster.

Simultáneamente, la tasa de errores mostró un pico controlado de 0.2 inducido por los escenarios de falla (error400/500.yml), registrado de forma diferenciada por servidor tanto para errores 4xx como 5xx gracias a las dos series del widget de tasa de errores. La disipación rápida de este pico confirma que las métricas reflejan exclusivamente el tráfico sintético y no una degradación en cascada de los nodos [7].

A nivel de recursos, el consumo de RAM evidenció dos niveles estables: el Datadog Agent oscilando entre 110-128 MB y los contenedores Flask manteniéndose cerca de los 30 MB. Esta horizontalidad descarta la existencia de fugas de memoria (memory leaks). Destaca que el agente de telemetría representó la mayor carga de procesamiento del clúster (picos de CPU del 30%), mientras que el balanceador y los backends mantuvieron una huella de cómputo mínima, indicando una amplia capacidad ociosa para escalar el tráfico de usuarios.

### C. Efectividad del Sistema de Alertas

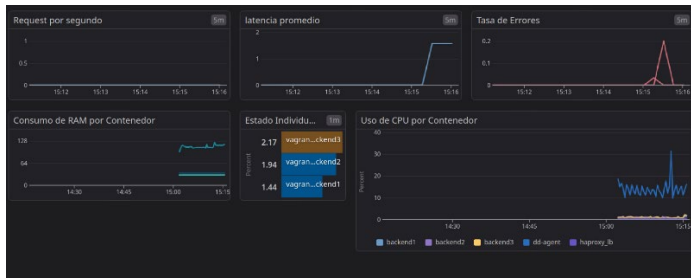
El monitoreo activo demostró ser superior a la inspección manual mediante dos alertas críticas. La primera, basada en la evaluación en tiempo real de errores del servidor ((haproxy.backend.response.5xx / haproxy.frontend.requests.rate) \* 100 > 5), superó con éxito las pruebas de validación al activarse de forma oportuna. La

segunda, configurada como un detector de caídas mediante la métrica `docker.containers.running` con umbral crítico menor a 1 por imagen (`vagrant_backend1`, `vagrant_backend2`, `vagrant_backend3`), con la opción `show_and_notify_no_data` activa, permitió identificar en segundos la detención arbitraria de un contenedor, enviando una notificación asíncrona precisa con el identificador del nodo afectado [5], [6].

#### D. Escalabilidad de la Arquitectura Stateless

La delegación de la persistencia hacia el nodo storage vía SFTP comprobó la eficacia de una arquitectura stateless (sin estado) en los backends. Al no conservar archivos localmente, se eliminó la necesidad de configurar afinidad de sesión (sticky sessions) en HAProxy, garantizando la compatibilidad total con Round Robin.

Adicionalmente, la incorporación de NGINX como servidor de descargas nativo mitigó cuellos de botella históricos. Al liberar a Flask de la transmisión de archivos pesados (superiores a 200 MB), la capa de aplicación se limitó a operaciones ligeras de escritura SFTP, mientras NGINX asumió eficientemente el trabajo de I/O en lectura, maximizando el rendimiento general de la solución [8].



**Fig. 3.** Telemetría del clúster en Datadog durante las pruebas de estrés, evidenciando picos transitorios de latencia y error sin degradación de recursos.

## VIII. CONCLUSIONES

El presente trabajo demostró la viabilidad de construir una infraestructura distribuida de alta disponibilidad, con observabilidad centralizada y validación de rendimiento, utilizando exclusivamente herramientas de código abierto y plataformas SaaS, sobre un entorno virtualizado estrictamente reproducible.

La separación arquitectónica entre la capa de virtualización y la de contenerización cumplió con creces el objetivo de garantizar la consistencia en el despliegue. Asimismo, la implementación de un modelo *stateless* en los *backends*, delegando la persistencia a un nodo de almacenamiento dual (SFTP para escritura y NGINX para lectura), resolvió cuellos

de botella previos. Esta decisión técnica permitió a HAProxy operar con máxima eficiencia bajo el algoritmo *Round Robin*, facilitando el escalado horizontal dinámico sin requerir mecanismos complejos de afinidad de sesión.

En el ámbito de la observabilidad, la elección de Datadog demostró ser fundamental para minimizar la sobrecarga operativa. Su capacidad de integración nativa y análisis en tiempo real superó el enfoque tradicional de múltiples herramientas segregadas. Las pruebas de estrés ejecutadas confirmaron no solo la estabilidad de los contenedores bajo cargas sostenidas y picos abruptos, sino también la eficacia del sistema de alertas automatizadas frente a caídas de nodos y degradaciones del servicio [5].

Finalmente, como trabajo futuro se identifican tres líneas de evolución para el sistema: (1) la incorporación de cifrado TLS/SSL en tránsito mediante certificados autogestionados (ej. Let's Encrypt); (2) la evaluación de algoritmos de balanceo dinámicos como *Least Connections* para cargas con tiempos de respuesta asimétricos; y (3) la implementación de un segundo nodo HAProxy bajo un protocolo de redundancia (VRRP), eliminando así el actual punto único de fallo (*Single Point of Failure*) de la arquitectura [1], [2].

## REFERENCIAS

- [1] M. D. Butcher, "An Introduction to HAProxy and Load Balancing Concepts," *DigitalOcean Community Tutorials*, Oct. 2021. [Online]. Available: <https://www.digitalocean.com/community/tutorials/an-introduction-to-haproxy-and-load-balancing-concepts>. [Accessed: May 2026].
- [2] HAProxy Technologies, *HAProxy 2.6 Configuration Manual*, HAProxy Technologies LLC, 2023. [Online]. Available: <https://www.haproxy.org/download/2.6/doc/configuration.txt>. [Accessed: May 2026].
- [3] HashiCorp, "Vagrant Documentation: Provisioning," *HashiCorp Developer*, 2024. [Online]. Available: <https://developer.hashicorp.com/vagrant/docs/provisioning>. [Accessed: May 2026].
- [4] Docker Inc., "Docker Compose Overview," *Docker Documentation*, 2024. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: May 2026].
- [5] Datadog Inc., "HAProxy Integration," *Datadog Documentation*, 2024. [Online]. Available: <https://docs.datadoghq.com/integrations/haproxy/>. [Accessed: May 2026].

[6] Datadog Inc., "Log Management," *Datadog Documentation*, 2024. [Online]. Available: <https://docs.datadoghq.com/logs/>. [Accessed: May 2026].

[7] Artillery.io, "Artillery Documentation: Load Testing," *Artillery Docs*, 2024. [Online]. Available: <https://www.artillery.io/docs>. [Accessed: May 2026].

[8] NGINX Inc., "NGINX Documentation: Serving Static Content," *NGINX Docs*, 2024. [Online]. Available: [https://nginx.org/en/docs/http/nginx\\_http\\_core\\_module.html](https://nginx.org/en/docs/http/nginx_http_core_module.html). [Accessed: May 2026].

[9] Pallets Projects, "Flask Documentation," *Flask*, 2024. [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: May 2026].